

System Architecture

1. Core Goals

The framework is built to support multiple sports (like Volleyball or Headball) without the need to rewrite the entire system every time a new one is added.

- **Open for Extension:** New sports can be added easily without changing the core engine's code.
- **Separation of Concerns:** Sport rules, visuals (UI), and game logic are kept separate to prevent bugs and messy code.
- **High Extensibility:** Using standard Java tools (interfaces and abstract classes), new sports can be "plugged in" seamlessly to the existing structure.

2. Layered Architecture

The system follows a rigid four-layer structure to ensure modularity and facilitate independent development of game components.

2.1 UI Layer

The UI Layer is entirely decoupled from the business logic, interacting exclusively with the ISport interface to maintain sport-agnosticism. This layer must support specific data-binding requirements, including the rendering of team logos and the management of AI-generated assets, such as male and female names for players and coaches loaded from external sources.

2.2 Game Logic Layer

This layer governs the simulation lifecycle and competition organization through three primary components: the MatchEngine for outcome calculation, the League for automated fixture generation and standings management, and the Match class for tracking individual event execution and scores.

2.3 Sport Layer

The Sport Layer serves as the rule-set abstraction, centered on the ISport interface. This interface acts as a mandatory contract, forcing concrete implementations like FootballSport and HeadballSport to define critical parameters via the getRequiredPlayers(), getMatchDuration(), and getPointForWin() methods.

2.4 Data Layer

This layer defines the persistence hierarchy and entity state. The BasePlayer abstract class serves as the entity foundation, encapsulating shared attributes including age, fitness, and injury status. Conversely, the BaseTeam class acts as a container for these entities, managing the teamName, a collection of players, and accumulated league points.

3. Core Class Connections

- ISport: This interface mandates the retrieval of core rules like winning points and match duration, the match engine runs independently of the sport type.

- **MatchEngine:** This component executes the `simulateMatch(team1, team2, rules)` method to calculate results by synthesizing data from the sport-specific rules and the attributes of the competing `BaseTeam` entities.
- **BasePlayer:** This abstract class centralizes the management of player health and progression, providing the core functional logic for the `train()` and `recover()` cycles used throughout the game loop.

4. UML Coordination Note The following technical directives must be strictly followed during the generation of the system class diagram:

- **Interfaces:** The `ISport` interface must be designated with the `<<interface>>` stereotype and include explicit labels for its abstract methods to denote the contract requirements.
- **Abstract Classes:** The `BasePlayer` class must be visually identified as abstract (via italics or labeling) to indicate it serves solely as a template for sport-specific player types.
- **Realization:** Represent the relationship between concrete implementations (e.g., `FootballSport`, `HeadballSport`) and the `ISport` interface using dashed realization arrows.
- **Inheritance:** Use solid arrows with empty triangular heads to show inheritance from sport-specific player classes (e.g., `FootballPlayer`) to the `BasePlayer` base class.
- **Composition/Aggregation:** The diagram must illustrate that the `League` aggregates `BaseTeam` instances and maintains a composition relationship with `Match` entities within its internal fixtures attribute.